BioSpecInfo

Interface Reference — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v219** · commit `f39b42c`

Generato il 2026-10-16

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Interface Reference — BioSpecInfo

Field	Value
Software	BioSpecInfo
Version described	`bsi-v219`
Purpose	Document the project's HTTP interfaces and the external APIs the application consumes.

0. A necessary premise

BioSpecInfo exposes no API. It is an entirely client-side Progressive Web App, served as static files: there is no backend, no application server, no REST endpoints belonging to the product.

This document would therefore be empty, were it not for two interfaces that do exist and that an assessor is entitled to see documented:

	What it is	Documented in
Spectra proxy	An **optional** Cloudflare Worker, owned by the user, that forwards calls to the AI providers. It is the only HTTP interface written in this project.	§1, §2
External APIs consumed	Public third-party services the application queries, always optionally.	§3
Internal JavaScript APIs	The extension points of the code, for whoever has to work on it.	§4

> An OpenAPI specification of non-existent endpoints would be plausible and false documentation. Only what exists is described here.

1. Spectra proxy — overview

The proxy solves two problems that have no other solution from the browser:

1. **CORS.** Several AI providers do not send the headers that allow a web page

to call them directly. From the Worker the call originates from a server and the constraint does not apply.

1. **Custody of the keys.** With the proxy the API key lives among the Worker's

secrets and not on the user's device.

It is **optional**: without a proxy the application works using the keys the user enters locally.

Deploy

```
cd proxy
npx wrangler login
npx wrangler deploy
npx wrangler secret put GROQ_KEYS     # one or more keys, comma-separated
```

Configuration

Variable	Type	Default	Description
`ORIGINI`	secret/variable	*(empty = all)*	List of permitted origins, comma-separated. **Leaving it empty makes the proxy open to anyone:** not advisable.
`LIMITE_IP`	variable	`20`	Requests per single IP within the time window
`TETTO_GIORNO`	variable	`2000`	Total daily requests
`GROQ_KEYS`, `GEMINI_KEYS`, `ANTHROPIC_KEYS`, `OPENAI_KEYS`, `XAI_KEYS`, `NVIDIA_KEYS`, `ZAI_KEYS`, `DEEPSEEK_KEYS`	secrets	—	API keys. Several keys comma-separated: when the first one's quota is exhausted (429), the proxy moves to the next **within the same request**.

> The variable names are kept in Italian because that is what they are called in the Worker: translating them here would document a configuration that does not exist.

Endpoints

Method	Path	Description
`OPTIONS`	`/*`	CORS preflight
`GET`	`/stato`	Which providers have a key configured
`GET` `POST`	`/{provider}/{path...}`	Forwarding to the provider

Recognised providers: `anthropic` · `gemini` · `groq` · `xai` · `nvidia` · `zai` · `openai` · `deepseek`

Status codes

Code	Meaning
`200`	The provider's answer, forwarded as it is
`204`	CORS preflight accepted
`400`	Request body unreadable
`403`	Origin not permitted by the `ORIGINI` list
`404`	Unknown provider
`405`	Method other than GET, POST or OPTIONS
`429`	Per-IP limit or daily ceiling exceeded

Code	Meaning
`503`	No key configured for that provider

Security choices

- **No header is forwarded blindly.** Only `anthropic-version` and

`anthropic-beta` pass through; any credential sent by the client stays out. A proxy that forwards everything is an open relay.

- **The `key` parameter is not propagated.** If a client puts it in the query

string, the proxy discards it: the key is the server's.

- **The body is read once and kept**, because retrying with a different key

requires being able to send it again — and a stream is consumed.

---

2. OpenAPI 3.0 specification

> The `description` fields are translated here; the **error messages in the examples are left in Italian**, because that is what the proxy actually returns to a user of the Italian application. Translating them would document responses that do not exist.

```yaml
openapi: 3.0.3
info:
  title: Spectra Proxy
  version: "1.0.0"
  description: |
    Optional proxy (Cloudflare Worker) for calls to language-model providers.
    It removes CORS blocks and keeps the API keys on the server rather than on
    the user's device.

    It is not part of the BioSpecInfo application: it is published by the user
    on their own Cloudflare account, and the application uses it only if it
    knows its address.
  license:
    name: Proprietary — All rights reserved (Samuele Pio Provenzano)
servers:
  - url: https://{workername}.{account}.workers.dev
    description: The user's instance
    variables:
      workername: { default: spectra-proxy }
      account:    { default: example }

tags:
  - name: Status
    description: Querying the configuration without revealing secrets
  - name: Forwarding
    description: Forwarding requests to the model providers

paths:
  /stato:
    get:
      tags: [Status]
      summary: Configured providers
      description: |
        Lists the providers for which the proxy holds at least one key.
        It reveals no key. The application uses it to show only the models that
        actually work, instead of letting them fail one by one.
      responses:
        "200":
          description: List of active providers
          content:
            application/json:
              schema:
                type: object
                required: [fornitori]
                properties:
                  fornitori:
                    type: array
                    items:
                      type: string
                      enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
              examples:
                twoKeys:
                  summary: Two providers configured
                  value: { fornitori: [groq, gemini] }
                none:
                  summary: Worker published but with no secrets
                  value: { fornitori: [] }
        "403":
          $ref: "#/components/responses/OrigineNonAmmessa"
```

```
/{fornitore}/{percorso}:
  parameters:
    - name: fornitore
      in: path
      required: true
      description: Destination provider
      schema:
        type: string
        enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
    - name: percorso
      in: path
      required: true
      description: |
        Path of the endpoint at the provider, reassembled as it is.
        Examples: `openai/v1/chat/completions` for Groq,
        `v1beta/models/gemini-3.6-flash:streamGenerateContent` for Gemini.
      schema: { type: string }
      example: openai/v1/chat/completions
  post:
    tags: [Forwarding]
    summary: Forward a request to the provider
    description: |
      The body is transmitted unchanged. The proxy adds the key in the way
      that provider requires (an `Authorization: Bearer` header, `x-api-key`,
      or a query parameter) and, if the call fails for exhausted quota,
      retries with the next key within the same request.
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            description: Body expected by the provider's API, transmitted unchanged
          examples:
            groq:
              summary: Streaming completion on Groq
              value:
                model: llama-3.3-70b-versatile
                stream: true
                messages:
                  - role: user
                    content: Which IR bands identify a carboxylic acid?
    responses:
      "200":
        description: |
          The provider's answer, forwarded without modification (typically
          `text/event-stream` when `stream: true`).
        content:
          application/json:
            schema: { type: object }
          text/event-stream:
            schema: { type: string }
      "400": { $ref: "#/components/responses/CorpoIlleggibile" }
      "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
      "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
      "429": { $ref: "#/components/responses/TroppeRichieste" }
      "503": { $ref: "#/components/responses/ChiaveAssente" }
  get:
    tags: [Forwarding]
```

```
      summary: Forward a read request to the provider
      description: |
        It serves, among other things, the resolution of the Gemini model, which
        queries `ListModels` with GET and could not pass without a key in the
        browser.
      responses:
        "200":
          description: The provider's answer
          content:
            application/json:
              schema: { type: object }
        "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
        "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
        "429": { $ref: "#/components/responses/TroppeRichieste" }
        "503": { $ref: "#/components/responses/ChiaveAssente" }

components:
  schemas:
    Errore:
      type: object
      required: [errore]
      properties:
        errore:
          type: string
          description: Message in Italian, meant to be shown to the user
  responses:
    OrigineNonAmmessa:
      description: The request's origin is not in the `ORIGINI` list
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Origine non ammessa da questo proxy." }
    FornitoreSconosciuto:
      description: The first path segment does not match a known provider
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Fornitore sconosciuto: pippo" }
    CorpoIlleggibile:
      description: The request body could not be read
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Corpo della richiesta illeggibile." }
    TroppeRichieste:
      description: The per-IP limit or the daily ceiling has been exceeded
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Troppe richieste da questo indirizzo: riprova fra un minuto." }
    ChiaveAssente:
      description: The proxy has no key configured for that provider
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example:
            errore: "Il proxy non ha una chiave per groq. Impostala con: wrangler secret put
GROQ_KEYS"

security: []
```

```
# CLIENT authentication is deliberately absent: the access control is the
# ORIGINI list plus the per-IP limits. Adding a client-side token would mean
# writing it into a static page, that is, making it public.
```

3. External APIs consumed by the application

All **optional**: with no network the application stays fully functional on local data, and says so to the user.

Service	Endpoints used	What it is for	If it does not answer

PubChem PUG-REST	`/compound/name/{name}/property/...`
	 `/compound/smiles/{smiles}/SDF?record_type=3d`
	 `/compound/cid/{cid}/PNG`
		IUPAC name, formula, CAS, GHS pictograms, 3D conformers, 2D images	The app uses the local tabulated data and says so

RCSB PDB	`https://files.rcsb.org/download/{pdb}.pdb`	Protein structures for the 3D viewer	Explicit message in the viewer
NASA / ESA	Public images and data	Astrochemistry module	The module shows the tabulated data
AI providers	At the user's choice, or via the proxy	Assistant answers	The assistant declares it and offers a reachable alternative

None of these calls transmits personal data. There are no persistent identifiers and no active telemetry (`BSI_TELEMETRY_URL` is empty).

> This is not asserted from the architecture alone: `audit_rete` seeds a canary value into the user's own data, uses the application across 6 pages and 87 sections, and inspects the URL, headers and body of every outgoing request.

4. Internal JavaScript APIs

Extension points for whoever has to work on the code. They are globals on `window` because the application is a set of classic scripts, not ES modules.

`window.BSISpettri` — spectral engine

Function	Signature	Description
`gruppi`	`(smiles, callback)`	Recognises functional groups via SMARTS/RDKit. Calls back with `null` if RDKit is unavailable: the caller decides whether to fall back or declare that it cannot compute.
`gruppiSync`	`(smiles) → object | null`	Synchronous version: answers only if the molecule is already cached
`bandeDaGruppi`	`(groups) → band[]`	Table of IR bands `{n, pk, w, i, f}`
`svgIR`	`(bands, options) → string`	IR spectrum in transmittance, SVG, deterministic
`legendaIR`	`(bands) → string`	Assignments in HTML
`quandoPronto`	`(callback)`	Notifies when the analysis of a molecule is available, so it can be redrawn
`precarica`	`()`	Warms up RDKit while nobody is waiting

`window.bsiProxy` — proxy state

Function	Description
`url()`	Configured address, empty string if absent
`stato()`	Promise: queries `/stato` and records which providers are covered
`copre(providerId)`	Whether that provider is covered by the proxy
`ricarica()`	Re-reads the configuration

Other notable points

Symbol	Description
`window.bsiLoadRDKit(cb)`	Loads RDKit once only and calls back with the instance
`window.irBandList(smiles)`	IR bands — replaced by the new engine, keeps the original signature
`window.makeIRsvg(bands, colour)`	IR drawing — likewise
`window.nmrPeakList(smiles)`	^{1}H peaks with integration
`window.bsiCancellaDati(groups)`	Selective deletion of the saved data
`window.bsiOpenUpdate()`	Opens the updates window
`window.bsiAccentoLeggibile(colour, background, threshold)`	Adjusts an accent so it reaches the WCAG threshold on that surface, preserving the hue
`window.bsiEtichettaLeggibile(background, threshold)`	The dual: picks the text colour from the background and, if neither extreme suffices, darkens the background while preserving the hue

Document updated to version `bsi-v219`.